Module II : File Systems and Device Management

- File Systems: Overview, File Organization and Access, File Directories, File Sharing, Record Blocking, Secondary Storage Management, File System Security,
- UNIX File System, Linux Virtual File System, Windows File System, Case studies of advanced file systems (ext4, ZFS, Btrfs).
- Device Management: I/O Devices, Organization of the I/O Function, Operating System Design Issues, I/O Buffering, I/O hub, Disk Scheduling Policies: FIFO, PRIORITY, SSTF, SCAN, C-SCAN, Linux I/O, Windows I/O.

OVERVIEW: Files and File Systems

- From the user's point of view, one of the most important parts of an operating system is the file system.
- The file system provides the resource abstractions typically associated with secondary storage.
- The file system permits users to create data collections, called files, with desirable properties, such as:
  - Long-term existence: Files are stored on disk or other secondary storage and do not disappear when a user logs off.
  - Sharable between processes: Files have names and can have associated access permissions that permit controlled sharing.
  - Structure: Depending on the file system, a file can have an internal structure that is convenient for particular applications. In addition, files can be organized into hierarchical or more complex structure to reflect the relationships among files.

- File system provides a means to store data organized as files, but a collection of functions that can be performed on files. Typical operations include the following:
- Create: A new file is defined and positioned within the structure of files.
- Delete: A file is removed from the file structure and destroyed.
- Open: An existing file is declared to be “opened” by a process, allowing the process to perform functions on the file.
- Close: The file is closed with respect to a process, so that the process no longer may perform functions on the file, until the process opens the file again.
- Read: A process reads all or a portion of the data in a file.
- Write: A process updates a file, either by adding new data that expands the size of the file or by changing the values of existing data items in the file.

- File Structure:
- Field • Record • File • Database
- On UNIX and UNIX-like systems, the basic file structure is just a stream of bytes.
- For example, a C program is stored as a file but does not have physical fields, records, and so on.

- FILE SYSTEM ARCHITECTURE:

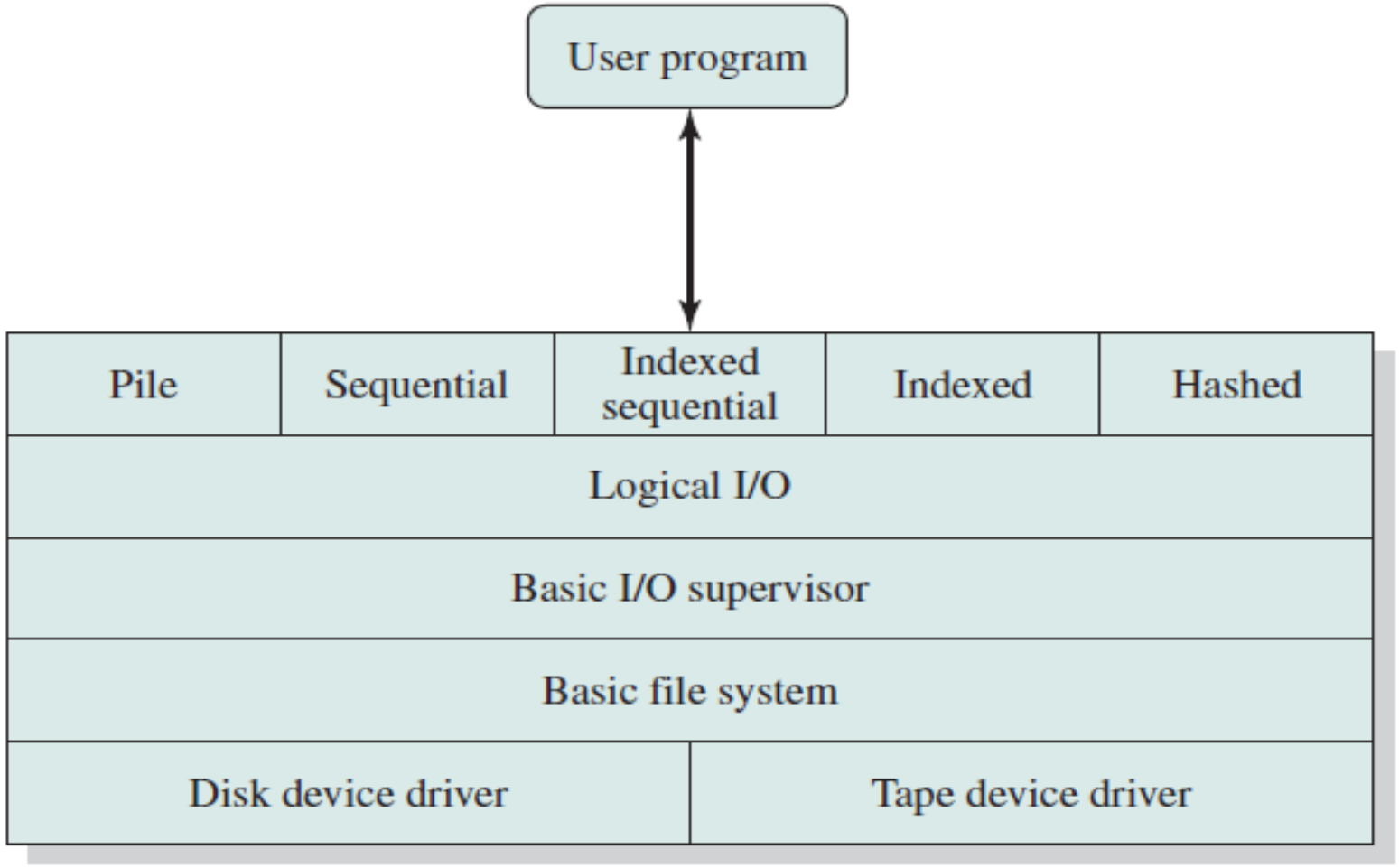


Figure 12.1 File System Software Architecture

- At the lowest level, device drivers communicate directly with peripheral devices. A device driver is responsible for starting I/O operations on a device and processing the completion of an I/O request.
- Device drivers are usually considered to be part of the operating system.
- The basic file system , or the physical I/O level. This is the primary interface with the environment outside of the computer system. It deals with blocks of data that are exchanged with disk or tape systems. Thus, it is concerned with the placement of those blocks on the secondary storage device and on the buffering of those blocks in main memory.

- The basic I/O supervisor is responsible for all file I/O initiation and termination. At this level, control structures are maintained that deal with device I/O, scheduling, and file status. The basic I/O supervisor selects the device on which file I/O is to be performed, based on the particular file selected. It is also concerned with scheduling disk and tape accesses to optimize performance.
- Logical I/O enables users and applications to access records. Thus, whereas the basic file system deals with blocks of data, the logical I/O module deals with file records. Logical I/O provides a general-purpose record I/O capability and maintains basic data about files.

- The level of the file system closest to the user is often termed the access method. It provides a standard interface between applications and the file systems and devices that hold the data. Different access methods reflect different file structures and different ways of accessing and processing the data.

- Another way of viewing the functions of a file system is shown in Figure 12.2 .
- Users and application programs interact with the file system by means of commands for creating and deleting files and for performing operations on files.
- Before performing any operation, the file system must identify and locate the selected file. This requires the use of some sort of directory that serves to describe the location of all files, plus their attributes.
- In addition, most shared systems enforce user access control: Only authorized users are allowed to access particular files in particular ways.
- The basic operations that a user or application may perform on a file are performed at the record level. The user or application views the file as having some structure that organizes the records, such as a sequential structure.

- I/O is done on a block basis. Thus, the records or fields of a file must be organized as a sequence of blocks for output and unblocked after input.
- To support block I/O of files, several functions are needed. The secondary storage must be managed. This involves allocating files to free blocks on secondary storage and managing free storage so as to know what blocks are available for new files and growth in existing files.

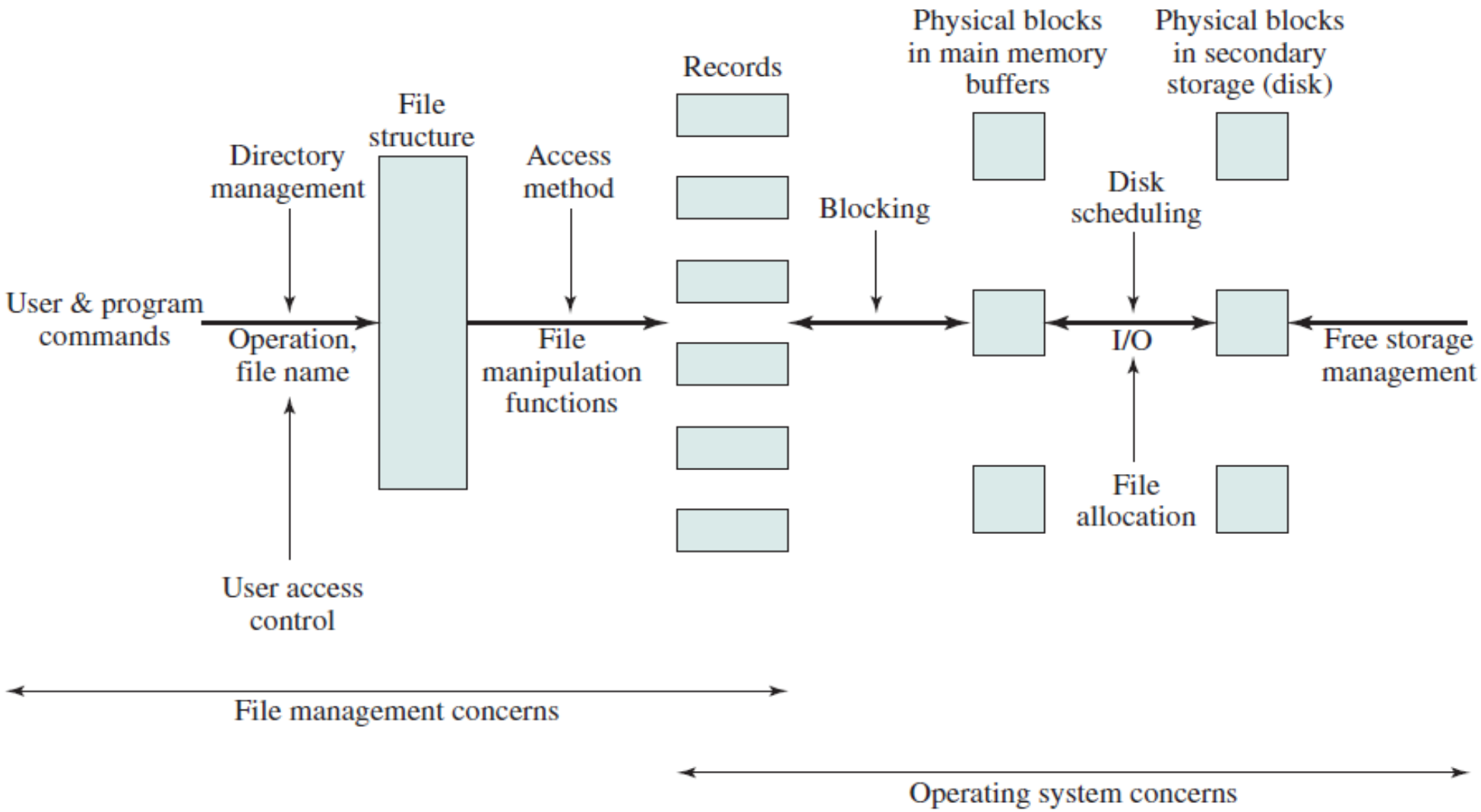


Figure 12.2 Elements of File Management

- FILE DIRECTORIES:
- Associated with any file management system and collection of files is a file directory. The directory contains information about the files, including attributes, location, and ownership.

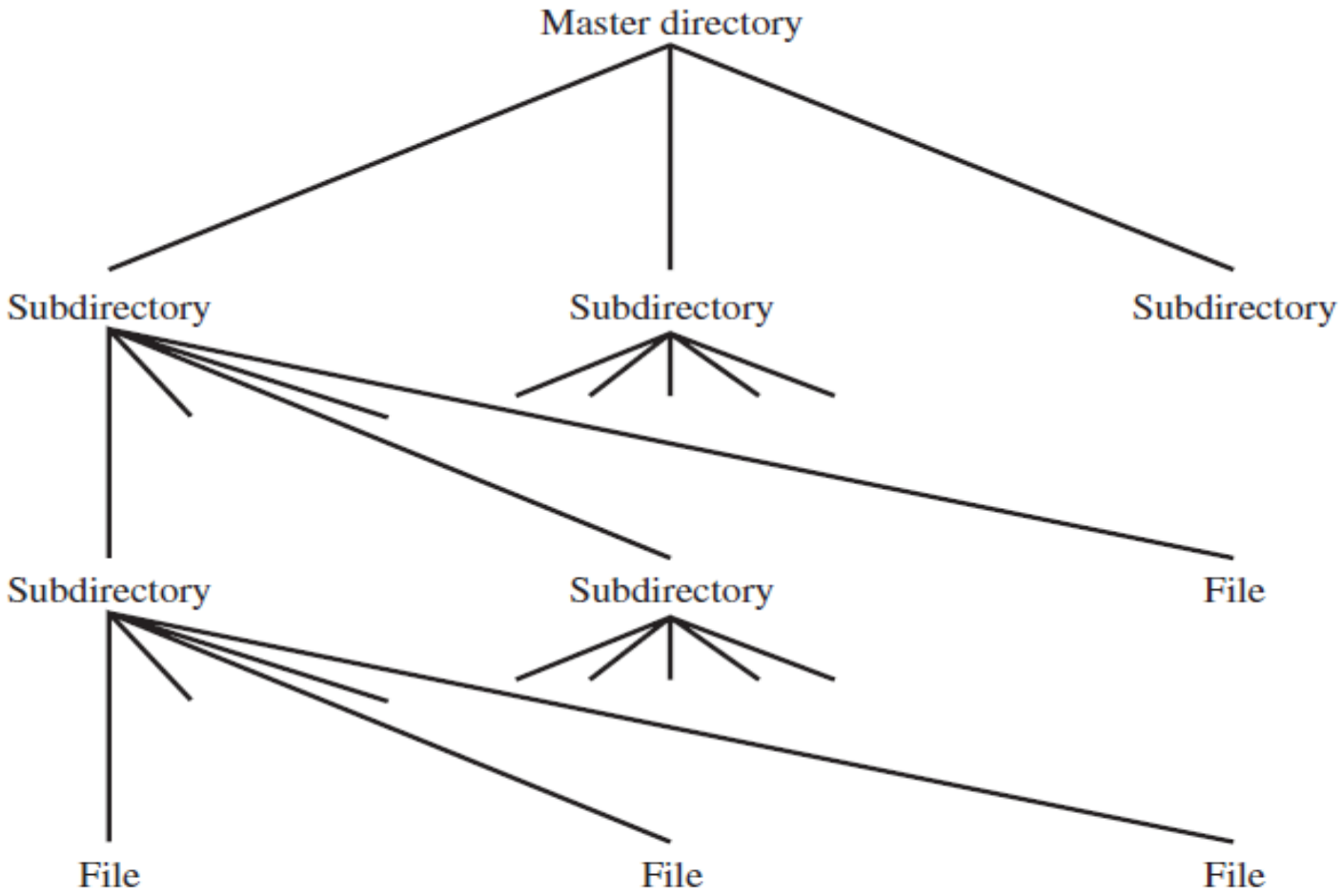


Figure 12.6 Tree-Structured Directory

- flexible approach to the directory structure is the hierarchical, or tree-structure, approach (Figure 12.6). As before,
- there is a master directory, which has under it a number of user directories. Each of these user directories, in turn, may have subdirectories and files as entries

- FILE SHARING:
- In a multiuser system, there is almost always a requirement for allowing files to be shared among a number of users. Two issues arise: access rights and the management of simultaneous access.
- The file system should provide a flexible tool for allowing extensive file sharing among users. The file system should provide a number of options so that the way in which a particular file is accessed can be controlled. Typically, users or groups of users are granted certain access rights to a file.
- One user is designated as owner of a given file, usually the person who initially
- created a file. The owner has all of the access rights listed previously and may grant rights to others. Access can be provided to different classes of users:
  - Specific user: Individual users who are designated by user ID
  - User groups: A set of users who are not individually defined. The system must have some way of keeping track of the membership of user groups.
  - All: All users who have access to this system. These are public files.

- RECORD BLOCKING:
- Records are the logical unit of access of a structured file, whereas blocks are the unit of I/O with secondary storage. For I/O to be performed, records must be organized as blocks.
- On most systems, blocks are of fixed length. This simplifies I/O, buffer allocation in main memory, and the organization of blocks on secondary storage.
- Second, what should the relative size of a block be compared to the average record size? The trade-off is this: The larger the block, the more records that are passed in one I/O operation. If a file is being processed or searched sequentially, this is an advantage, because the number of I/O operations is reduced by using larger blocks, thus speeding up processing. On the other hand, if records are being accessed randomly and no particular locality of reference is observed, then larger blocks result in the unnecessary transfer of unused records.

- Given the size of a block, there are three methods of blocking that can be used:
- Fixed blocking: Fixed-length records are used, and an integral number of records are stored in a block. There may be unused space at the end of each block. This is referred to as internal fragmentation.
- Variable-length spanned blocking: Variable-length records are used and are packed into blocks with no unused space. Thus, some records must span two blocks, with the continuation indicated by a pointer to the successor block.
- Variable-length unspanned blocking: Variable-length records are used, but spanning is not employed. There is wasted space in most blocks because of the inability to use the remainder of a block if the next record is larger than the remaining unused space.

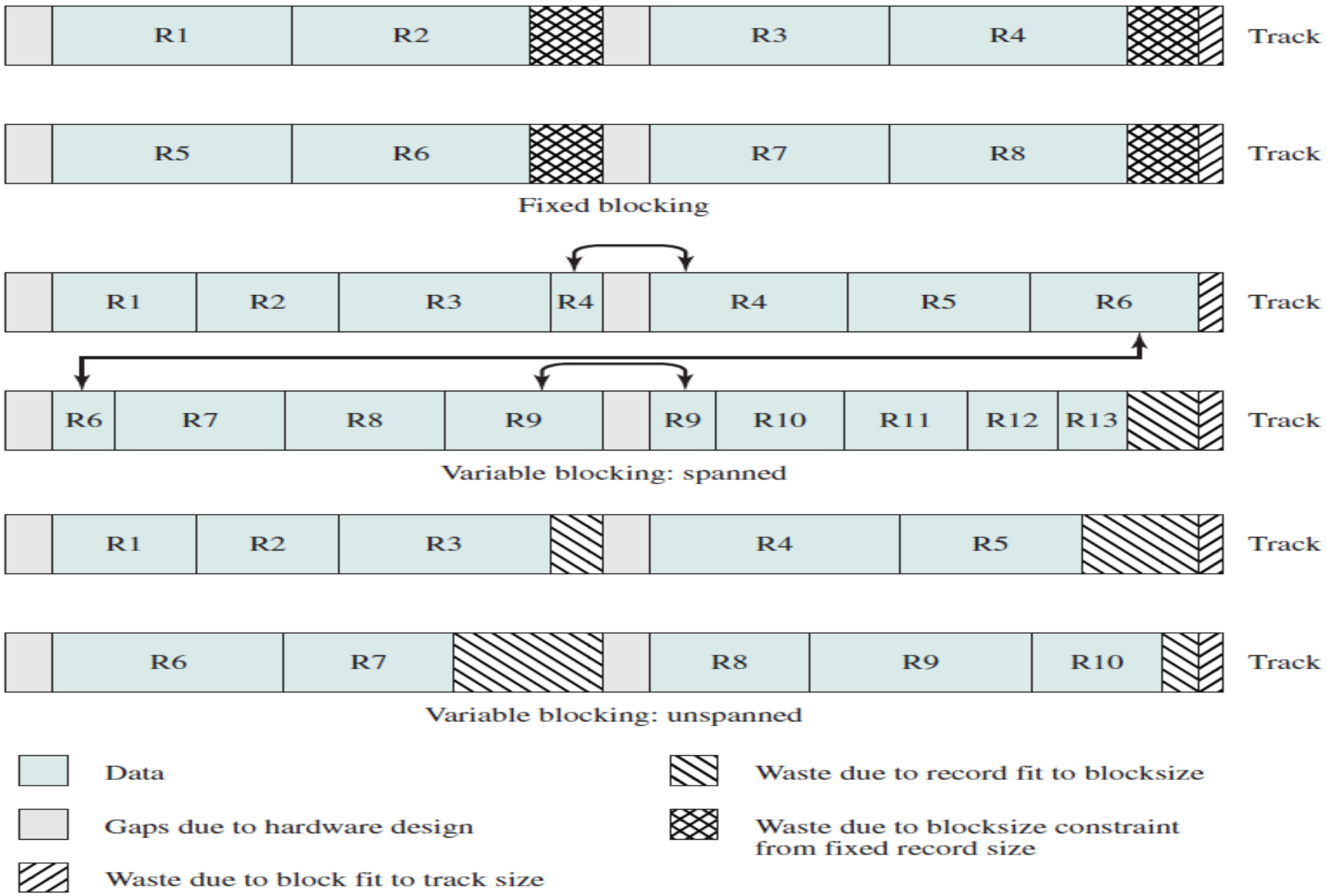


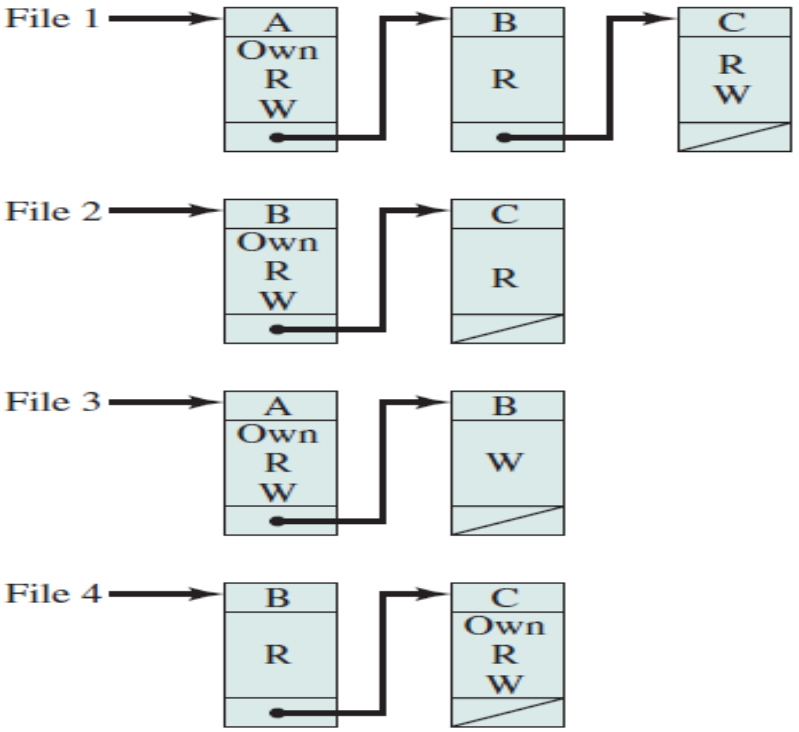
Figure 12.8 Record Blocking Methods [WIED87]

FILE SYSTEM SECURITY:

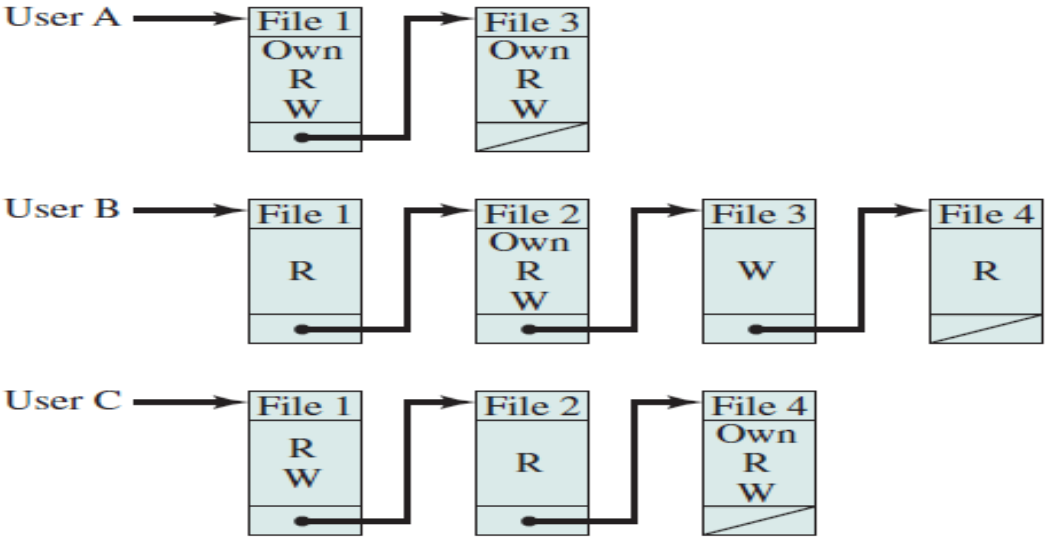
- Through the user–access control procedure, a user can be identified to the system. Associated with each user, there can be a profile that specifies permissible operations and file accesses. The operating system can then enforce rules based on the user profile.
- A general model of access control as exercised by a file management system is that of an access matrix as shown in the fig

	File 1	File 2	File 3	File 4	Account 1	Account 2
User A	Own R W		Own R W		Inquiry credit	
User B	R	Own R W	W	R	Inquiry debit	Inquiry credit
User C	R W	R		Own R W		Inquiry debit

(a) Access matrix



(b) Access control lists for files of part (a)



(c) Capability lists for files of part (a)

Figure 12.15 Example of Access Control Structures

- An access matrix is usually sparse and is implemented by decomposition in one of two ways. The matrix may be decomposed by columns, yielding access control lists (Figure 12.15b). Thus for each object, an access control list lists users and their permitted access rights. The access control list may contain a default, or public, entry. This allows users that are not explicitly listed as having special rights to have a default set of rights. Elements of the list may include individual users as well as groups of users.
- Decomposition by rows yields capability tickets (Figure 12.15c). A capability ticket specifies authorized objects and operations for a user. Each user has a number of tickets and may be authorized to loan or give them to others. Because tickets may be dispersed around the system, they present a greater security problem than access control lists.

- LINUX VIRTUAL FILE SYSTEM
- Linux includes a versatile and powerful file-handling facility, designed to support a wide variety of file management systems and file structures. The approach taken in Linux is to make use of a virtual file system (VFS) , which presents a single, uniform file system interface to user processes.
- The VFS defines a common file model that is capable of representing any conceivable file system's general feature and behavior. The VFS assumes that files are objects in a computer's mass storage memory that share basic properties regardless of the target file system or the underlying processor hardware.
- File has an owner, protection against unauthorized access or modification, and a variety of other properties. A file may be created, read from, written to, or deleted. For any specific file system, a mapping module is needed to transform the characteristics of the real file system to the characteristics expected by the virtual file system.

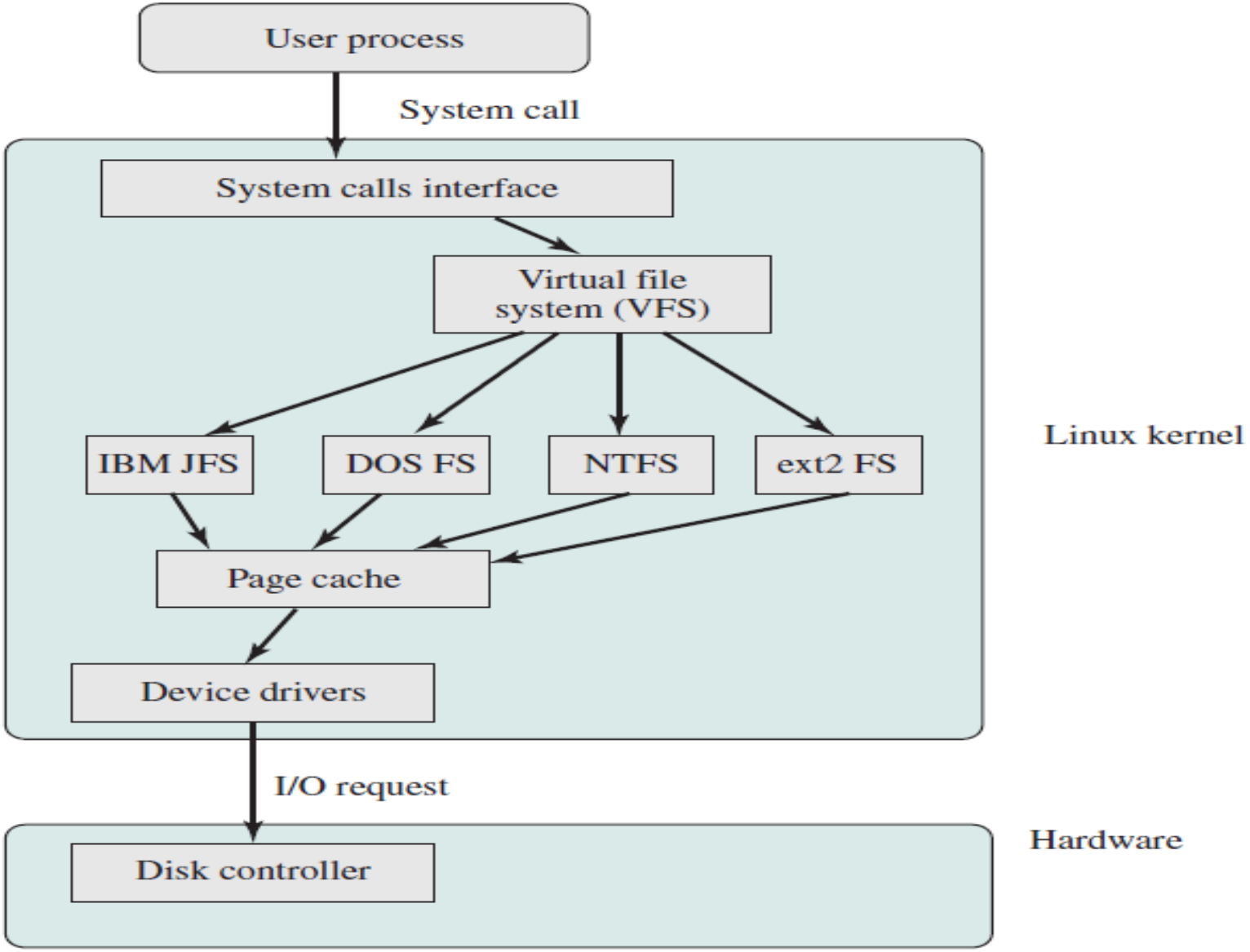


Figure 12.19 Linux Virtual File System Context

- The original user file system call is translated into a call that is native to the target file system. The target file system software is then invoked to perform the requested function on a file or directory under its control and secondary storage. The results of the operation are then communicated back to the user in a similar fashion.
- VFS is an object-oriented scheme. Because it is written in C, rather than a language that supports object programming (such as C++ or Java), VFS objects are
- implemented simply as C data structures. Each object contains both data and pointers
- to file-system-implemented functions that operate on data. The four primary
- object types in VFS are as follows:

- Superblock object: Represents a specific mounted file system
- Inode object: Represents a specific file
- Dentry object: Represents a specific directory entry
- File object: Represents an open file associated with a process.

The Superblock Object

The superblock object stores information describing a specific file system. The superblock corresponds to the file system superblock or file system control block, which is stored in a special sector on disk.

- UNIX FILE MANAGEMENT

- In the UNIX file system, six types of files are distinguished:
- Regular, or ordinary: Contains arbitrary data in zero or more data blocks. Regular files contain information entered in them by a user, an application program, or a system utility program. The file system does not impose any internal structure to a regular file but treats it as a stream of bytes.
- Directory: Contains a list of file names plus pointers to associated inodes (index nodes), described later. Directories are hierarchically organized Directory files are actually ordinary files with special write protection privileges so that only the file system can write into them, while read access is available to user programs.

- Special: Contains no data but provides a mechanism to map physical devices to file names. The file names are used to access peripheral devices, such as terminals and printers. Each I/O device is associated with a special file.
- Named pipes: A pipe is an inter process communications facility. A pipe file buffers data received in its input so that a process that reads from the pipe's output receives the data on a first-in-first-out basis.
- Links: In essence, a link is an alternative file name for an existing file.
- Symbolic links: This is a data file that contains the name of the file it is linked to.

- Inodes
- Modern UNIX operating systems support multiple file systems but map all of these into a uniform underlying system for supporting file systems and allocating disk space to files. All types of UNIX files are administered by the OS by means of inodes.
- An inode (index node) is a control structure that contains the key information needed by the operating system for a particular file.
- The attributes of the file as well as its permissions and other control information are stored in the inode.
- inode structure includes the following data elements:

- The type and access mode of the file
- The file's owner and group-access identifiers
- The time that the file was created, when it was most recently read and written, and when its inode was most recently updated by the system
- The size of the file in bytes
- A sequence of block pointers
- The number of physical blocks used by the file, including blocks used to hold indirect pointers and attributes
- The number of directory entries that reference the file
- The kernel and user-settable flags that describe the characteristics of the file
- The generation number of the file (a randomly selected number assigned to the inode each time that the latter is allocated to a new file; the generation number is used to detect references to deleted files)
- The blocksize of the data blocks referenced by the inode

- The size of the extended attribute information
- Zero or more extended attribute entries